From Linear Models to Neural Networks

Linear & logistic regression, neurons, MLPs, universal approximation

Prof. Nipun Batra

ES 667 · Deep Learning · IIT Gandhinagar

Lecture 1 chose the loss; Lecture 2 expands the prediction rule

Keep from Lecture 1

Regression output $\rightarrow$ Gaussian model $\rightarrow$ MSE

Class probabilities $\rightarrow$ Bernoulli / categorical model $\rightarrow$ cross-entropy

Change in Lecture 2

Start with one weighted sum, expose what it cannot represent, then let hidden layers learn nonlinear features.

$$\boldsymbol{x} \rightarrow f_{\boldsymbol{\theta}}(\boldsymbol{x}) \rightarrow \text{output parameters} \rightarrow \text{same Lecture 1 loss}$$

We will not re-derive the losses; today is about making f_θ more expressive.

The map for today

V VISUAL

One question drives the lecture: **what changes when one weighted sum becomes a network?**

weighted sum → expose the limitation → learn nonlinear features → compose them

We will calculate every stage for small numerical examples before writing the general matrix form.

One notation covers every prediction rule

Q QUESTION

Write $f_{\boldsymbol{\theta}}(\boldsymbol{x})$ for the model's prediction at input $\boldsymbol{x}$.

$\boldsymbol{x}$: the input features
$\boldsymbol{\theta}$: all trainable weights and biases
$\boldsymbol{f_{\theta}}$: the rule that maps input to a score or prediction

Starting point — one weighted sum plus a bias:

$$f_{\boldsymbol{\theta}}(\boldsymbol{x}) = \boldsymbol{w}^{\top}\boldsymbol{x} + b$$

We will build the network gradually; the full nested formula comes only after each part is concrete.

The linear starting point

Start with one weighted sum

For an input vector $\boldsymbol{x} \in \mathbb{R}^d$ and scalar target y,

$$\hat{y} = \boldsymbol{w}^\top \boldsymbol{x} + b$$

For $\boldsymbol{x} = (2, -1)$, $\boldsymbol{w} = (1.5, 0.5)$, and $b = -0.2$:

$$\hat{y} = 1.5(2) + 0.5(-1) - 0.2 = 2.3.$$

Regression: read 2.3 as the predicted mean.

Classification: read the weighted sum as a score, then convert it to probabilities.

The same weighted-sum core can feed different output models and losses from Lecture 1.

A whole batch is one matrix multiplication

For $\boldsymbol{X} \in \mathbb{R}^{n \times d}$ and $\boldsymbol{y} \in \mathbb{R}^{n}$,

$$\hat{\boldsymbol{y}} = \boldsymbol{X}\boldsymbol{w} + b\mathbf{1} = \tilde{\boldsymbol{X}}\boldsymbol{\theta}.$$

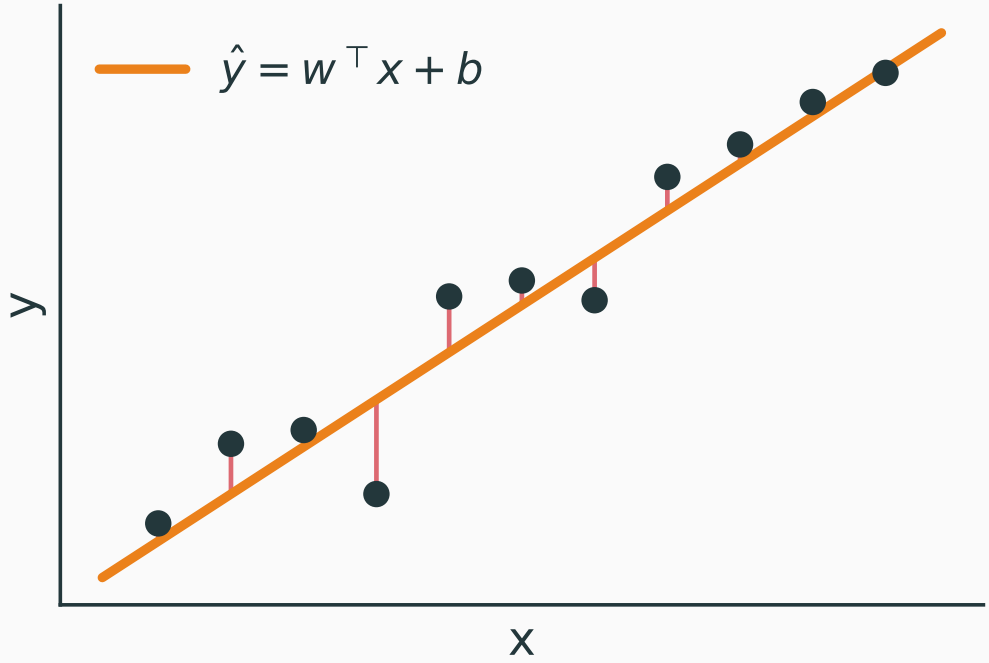
$$\tilde{\boldsymbol{X}} \in \mathbb{R}^{n \times (d+1)}, \quad \boldsymbol{\theta} \in \mathbb{R}^{d+1}, \quad \hat{\boldsymbol{y}} \in \mathbb{R}^{n}.$$

Rows are examples; columns are features; matrix multiplication performs all n predictions together.

For regression, one weighted sum gives one flat trend

V VISUAL

$$\hat{y} = \boldsymbol{w}^\top \boldsymbol{x} + b$$

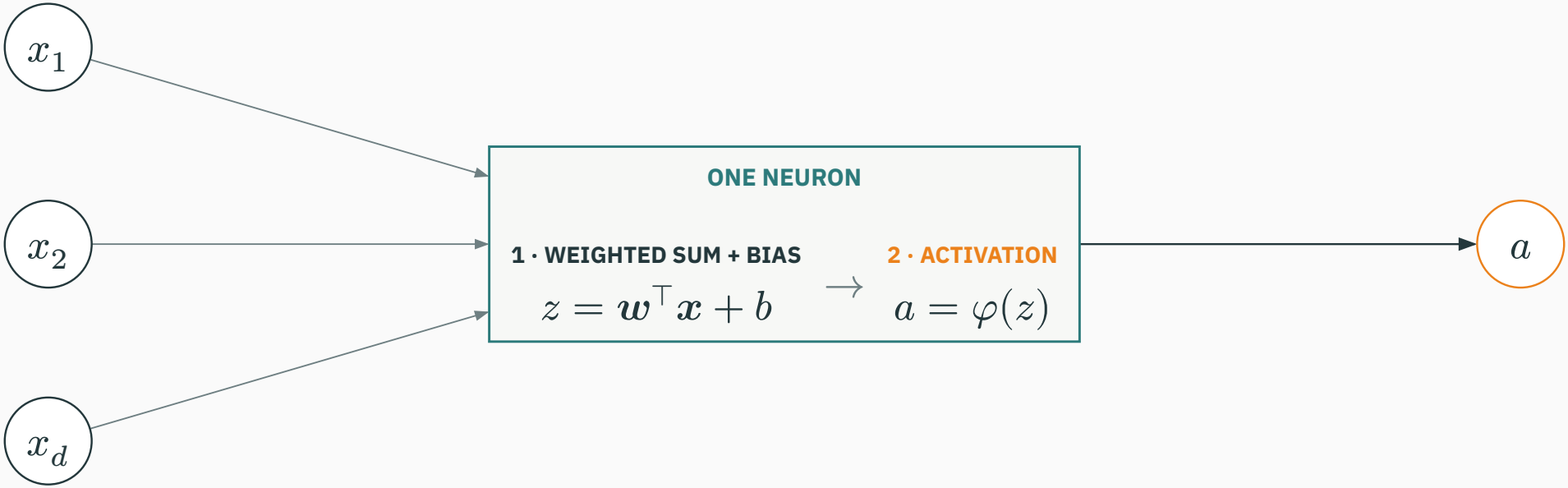


The model can tilt and shift a line or plane, but it cannot bend to follow nonlinear structure.

From a linear score to a neuron

A neuron has two stages

First compute a weighted sum plus bias; then pass that value through an activation.



One neuron recovers familiar Lecture 1 models

Let $z = \boldsymbol{w}^\top \boldsymbol{x} + b$. The output interpretation chooses the last step:

Regression
identity output
$\hat{y} = z$

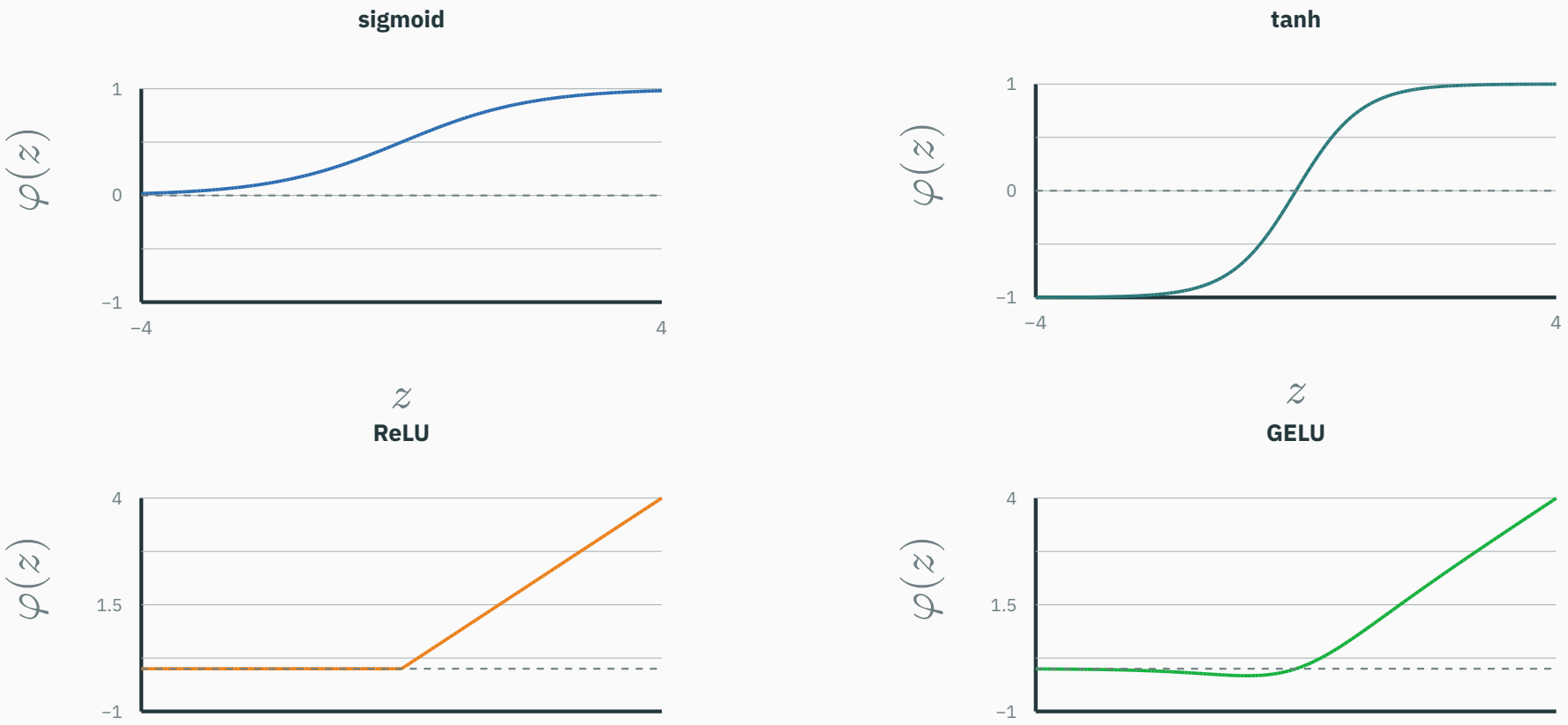
Binary class
sigmoid output
$p = \sigma(z)$

K classes
K scores + softmax
$\boldsymbol{p} = \mathrm{softmax}(\boldsymbol{z})$

Lecture 1 tells us how to interpret the output and choose the loss; Lecture 2 changes the features entering that output.

Common activation functions

V VISUAL



Numeric axes are visible; plots in each row share a y-scale for honest shape comparison.

Question: do more linear layers create new shapes?

Q QUESTION

Remove every activation. What can a stack of “weighted sum + bias” layers represent?

A. One equivalent weighted sum + bias

B. Any curved decision boundary

C. Only a constant function

D. A probability distribution automatically

Answer: line after line is still a line

A ANSWER

Algebraic view

$$h_1 = W_1 x + b_1$$

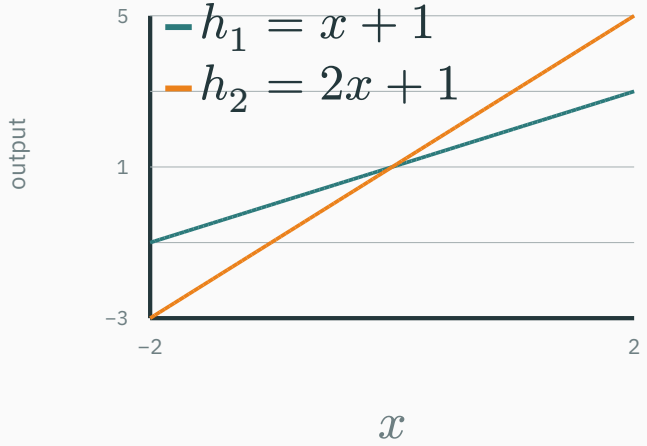
$$h_2 = W_2 h_1 + b_2$$

$$h_2 = \underbrace{W_2 W_1}_{\text{one matrix}} x + \underbrace{(W_2 b_1 + b_2)}_{\text{one bias}}$$

$W_2 W_1$ is one new weight matrix; the remaining term is one new bias vector.

Geometric view

$$h_1 = x + 1, \quad h_2 = 2h_1 - 1 = 2x + 1$$



The slope changes; no bend appears.

Correct: A. Without a nonlinear activation, extra layers still reduce to one weighted sum plus bias.

Interactive: activation functions

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/dl-teaching/interactives/activation-explorer.html

Compare activations and their **derivatives**, and see where gradients vanish.
Controls: activation type · input z · show derivative · compare saturation.

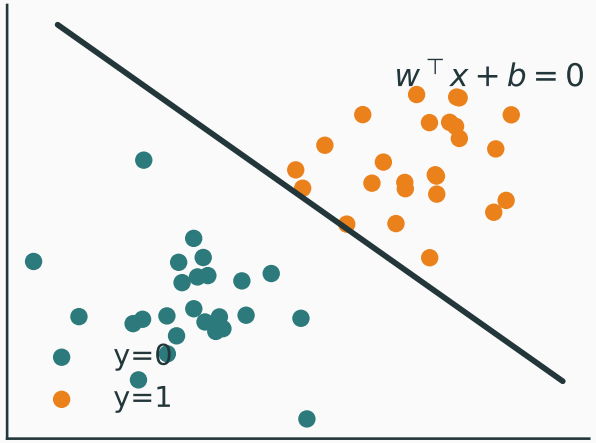
Explore large positive and negative logits, where sigmoid's derivative becomes tiny.

Where a single weighted sum fails

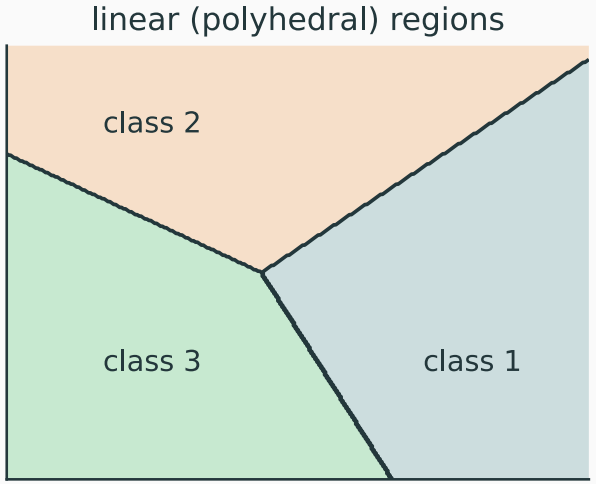
Single-layer classifiers draw only straight boundaries

V VISUAL

Binary: threshold one score $z = \boldsymbol{w}^\top \boldsymbol{x} + b$.



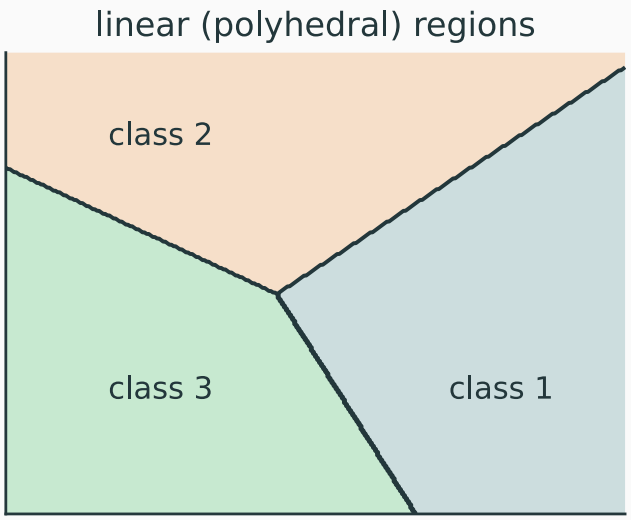
Multi-class: compare several linear scores.



Sigmoid and softmax change scores into probabilities; they do not bend the boundaries in input space.

Question: why are the multi-class boundaries straight?

Q QUESTION



three linear scores divide the input plane

The classifier predicts

$$\hat{y}(\boldsymbol{x}) = \arg\max_k p_{k(\boldsymbol{x})}.$$

When is a point on the boundary between predicted classes i and j?

A. Whenever $p_i = p_j$
B. When $p_i = p_j$ and both are maximal
C. Whenever $p_i + p_j = 1$
D. Only when all K probabilities are equal

Answer, part 1: equal probabilities give a flat tie set

A ANSWER

Softmax preserves score comparisons.

$$\frac{p_i}{p_j} = \frac{\exp(z_i)}{\exp(z_j)} = \exp(z_i - z_j).$$

Therefore

$$p_i = p_j \leftrightarrow z_i = z_j,$$

$$p_i > p_j \leftrightarrow z_i > z_j.$$

$\arg\max_k p_k = \arg\max_k z_k$: softmax changes the scale, not the ranking.

Linear-score ties are flat.

$$z_{k(\boldsymbol{x})} = \boldsymbol{w}_k^\top \boldsymbol{x} + b_k$$

$$z_i = z_j \leftrightarrow (\boldsymbol{w}_i - \boldsymbol{w}_j)^\top \boldsymbol{x} + (b_i - b_j) = 0.$$

This equation is a line in 2-D and a hyperplane in higher dimensions.

It is the **candidate pairwise boundary** between classes i and j.

Pairwise equality explains why every candidate boundary is straight.
But is every tie visible?

Answer, part 2: the tied classes must also be on top

A ANSWER

A pairwise tie alone is not enough.

Suppose

$\boldsymbol{z} = (1, 1, 3).$

Then

$p_1 = p_2 < p_3.$

The prediction is class 3. The point lies on $z_1 = z_2$, but **not** on a visible class-1/class-2 boundary.

Decision region for class i

$$\mathcal{R}_i = \{\boldsymbol{x} : z_i \geq z_l \quad \text{for every} \quad l\}.$$

Boundary shared by classes i and j

$$\mathcal{B}_{ij} = \mathcal{R}_i \cap \mathcal{R}_j$$

$$\mathcal{B}_{ij} = \{\boldsymbol{x} : z_i = z_j \geq z_l \quad \text{for every} \quad l\}.$$

Equivalently, $p_i = p_j$ and both probabilities are maximal.

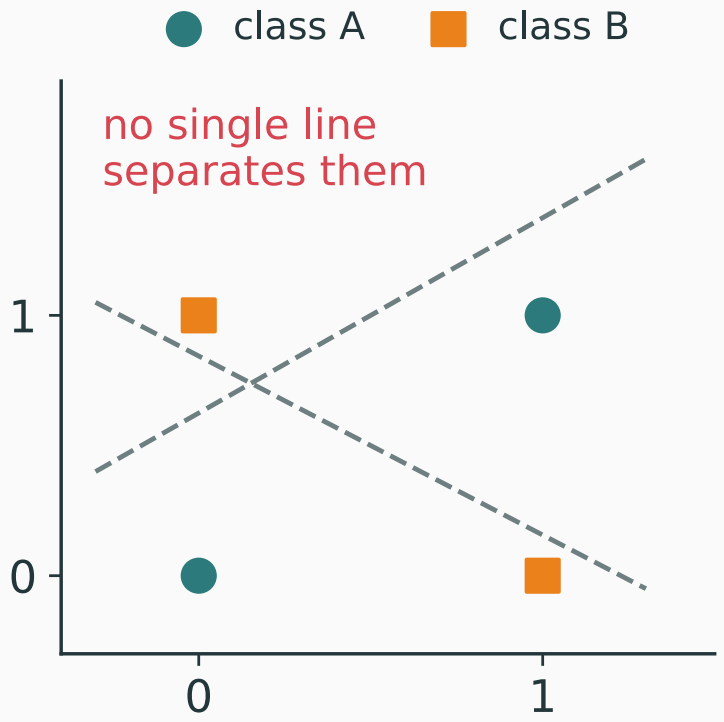
Correct: B. The visible boundary is the top-scoring portion of a pairwise linear tie set.

XOR exposes the missing ingredient

V VISUAL

No single line separates the two XOR classes.

The model first needs a transformation of the input—new features in which the classes become separable.



Hidden nonlinear units learn the transformation instead of asking us to hand-design it.

The multilayer perceptron

A hidden layer learns features before the output

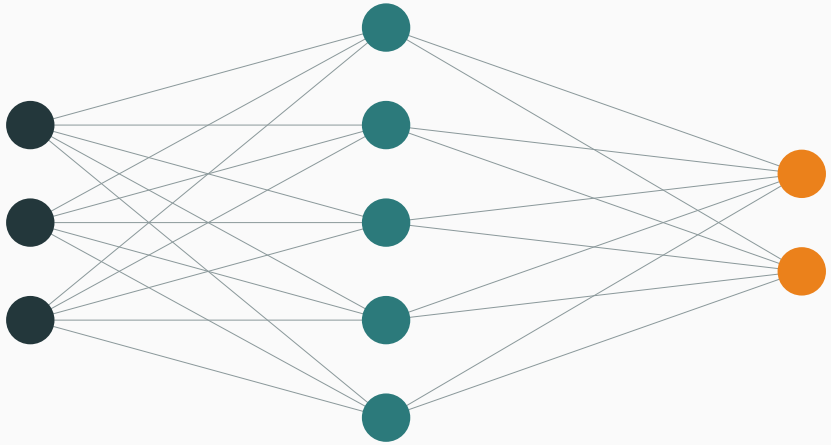
Instead of $\boldsymbol{z} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}$, insert a nonlinear layer first:

$$\boldsymbol{h} = \varphi(\boldsymbol{W}_1\boldsymbol{x} + \boldsymbol{b}_1), \quad \boldsymbol{z} = \boldsymbol{W}_2\boldsymbol{h} + \boldsymbol{b}_2$$

For classification, $\boldsymbol{p} = \mathrm{softmax}(\boldsymbol{z})$. This is a **one-hidden-layer MLP**.

Layer dimensions must agree

V VISUAL

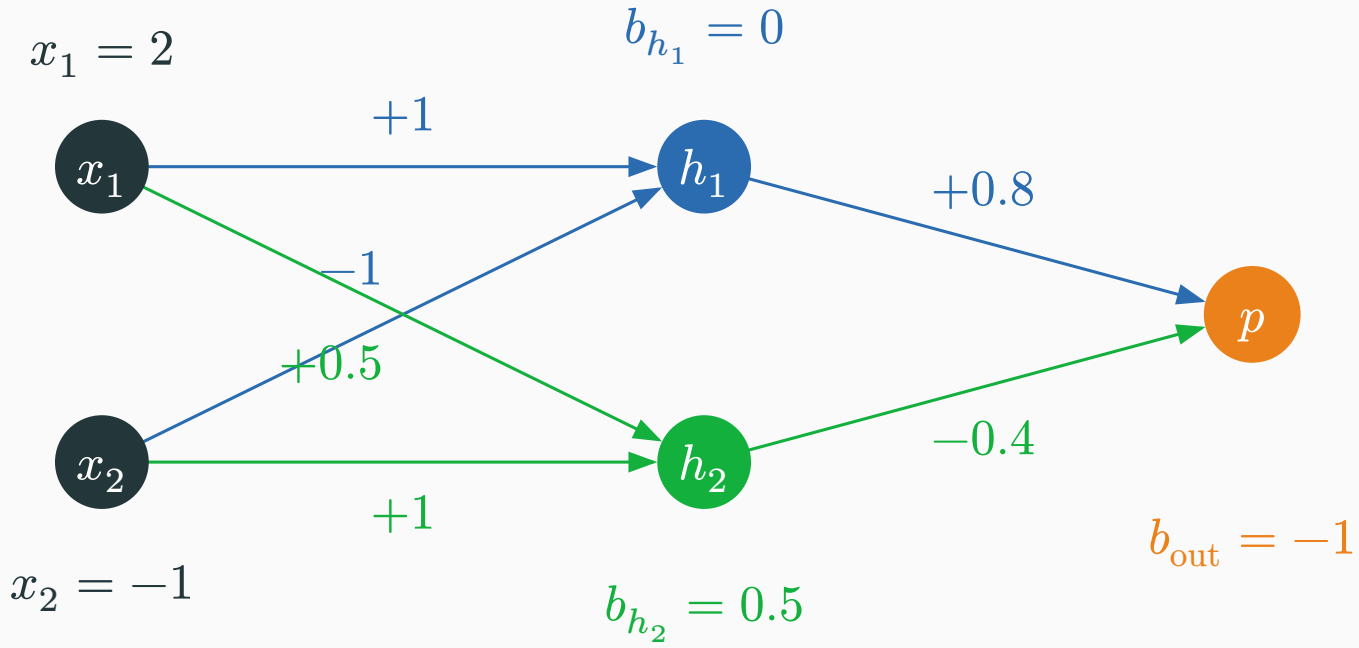


d inputs m hidden units K output scores

$$\boldsymbol{x} \in \mathbb{R}^d$$
$$\boldsymbol{W}_1 \in \mathbb{R}^{m \times d}, \quad \boldsymbol{b}_1 \in \mathbb{R}^m$$
$$\boldsymbol{h} \in \mathbb{R}^m$$
$$\boldsymbol{W}_2 \in \mathbb{R}^{K \times m}, \quad \boldsymbol{b}_2 \in \mathbb{R}^K$$
$$\boldsymbol{z} \in \mathbb{R}^K$$

First see the complete $2 \to 2 \to 1$ network

V VISUAL



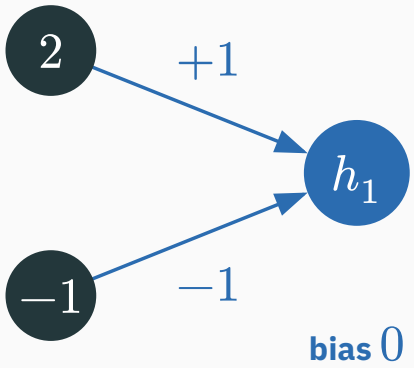
matrix row 1: $(1, -1)$ · **bias** 0 · **output** $+0.8$

matrix row 2: $(0.5, 1)$ · **bias** 0.5 · **output** -0.4

Arrow colours match the matrix rows and the output weights we will calculate.

Compute the blue hidden feature

D DERIVATION



Pre-activation

$$u_1 = (1 \;\; -1)\begin{pmatrix} 2 \\ -1 \end{pmatrix} + 0$$

$$u_1 = 1(2) + (-1)(-1) + 0$$

$$u_1 = 2 + 1 = 3.$$

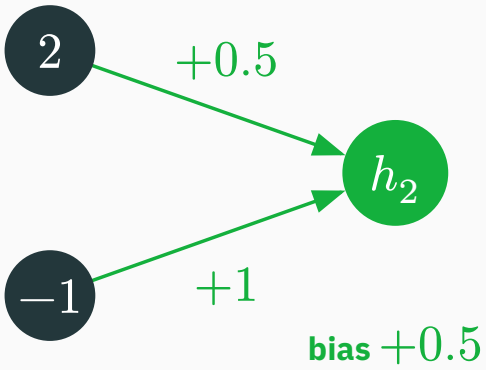
Activation

$$h_1 = \mathrm{ReLU}(u_1) = \max(0, 3) = 3.$$

The blue unit computes the concrete feature $h_1(x) = \mathrm{ReLU}(x_1 - x_2)$.

Compute the green hidden feature

D DERIVATION



Pre-activation

$$u_2 = (0.5 \;\; 1)\begin{pmatrix} 2 \\ -1 \end{pmatrix} + 0.5$$

$$u_2 = 0.5(2) + 1(-1) + 0.5$$

$$u_2 = 1 - 1 + 0.5 = 0.5.$$

Activation

$$h_2 = \mathrm{ReLU}(u_2) = \max(0, 0.5) = 0.5.$$

The green unit computes $h_2(x) = \mathrm{ReLU}(0.5x_1 + x_2 + 0.5)$**.**

Check both hidden calculations in matrix form

D DERIVATION

$$\boldsymbol{u} = \boldsymbol{W}_1 \boldsymbol{x} + \boldsymbol{b}_1$$

$$= \begin{pmatrix} 1 & -1 \\ 0.5 & 1 \end{pmatrix} \begin{pmatrix} 2 \\ -1 \end{pmatrix} + \begin{pmatrix} 0 \\ 0.5 \end{pmatrix}.$$

$u_1 = 1(2) + (-1)(-1) + 0 = 3$

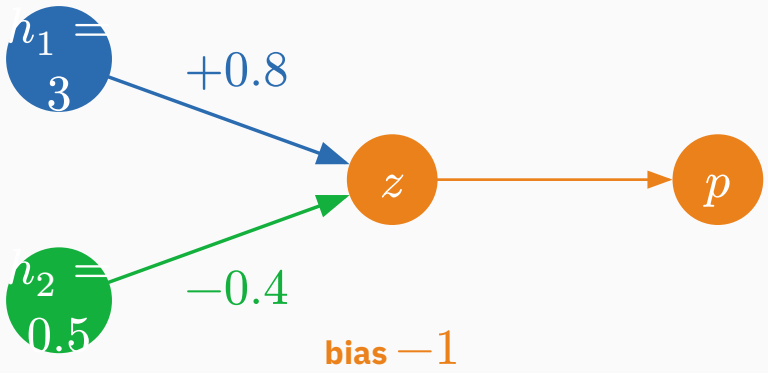
$u_2 = 0.5(2) + 1(-1) + 0.5 = 0.5$

$$\boldsymbol{h} = \mathrm{ReLU}(\boldsymbol{u}) = \begin{pmatrix} \max(0,3) \\ \max(0,0.5) \end{pmatrix} = \begin{pmatrix} 3 \\ 0.5 \end{pmatrix}.$$

Rows of W_1 are neurons: each row computes one pre-activation, then ReLU acts elementwise.

Combine the two features, then compute probability and loss

D DERIVATION



Score

$$z = \begin{pmatrix} 0.8 & -0.4 \end{pmatrix} \begin{pmatrix} 3 \\ 0.5 \end{pmatrix} - 1 = 0.8(3) - 0.4(0.5) - 1 = 1.2.$$

Probability

$$p = \sigma(1.2) = \frac{1}{1 + \exp(-1.2)} \approx 0.769.$$

Binary cross-entropy for $y = 1$

$$\mathcal{L} = -[1 \log p + 0 \log(1 - p)] = -\log(0.769) \approx 0.262.$$

$x \to u \to h \to z \to p \to \mathcal{L}$ **is one complete forward pass.**

What exact features did the two hidden units learn?

V VISUAL

BLUE FEATURE

$h_1(\boldsymbol{x}) = \mathrm{ReLU}(x_1 - x_2)$

Boundary: $x_2 = x_1$
Active side: $x_1 > x_2$

$h_1(2, -1) = 3$ **on**
$h_1(0, 1) = 0$ **off**

GREEN FEATURE

$h_2(\boldsymbol{x}) = \mathrm{ReLU}(0.5x_1 + x_2 + 0.5)$

Boundary: $x_2 = -0.5x_1 - 0.5$
Active side: points above this line

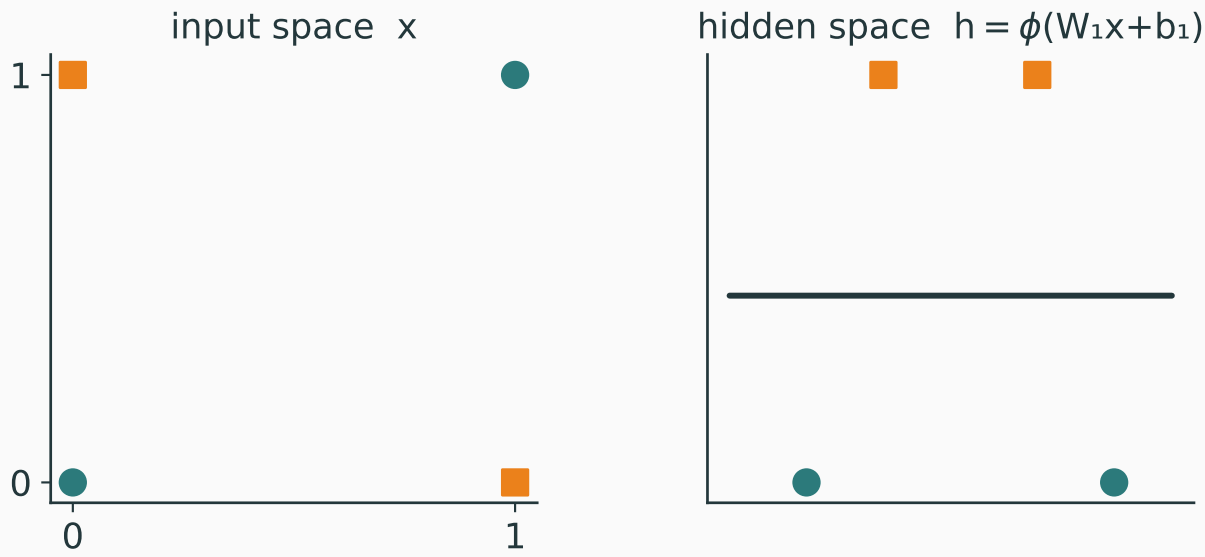
$h_2(2, -1) = 0.5$ **on**
$h_2(0, -1) = 0$ **off**

At $\boldsymbol{x} = (2, -1)$, the output receives $0.8h_1 = 2.4$ and $-0.4h_2 = -0.2$.

A learned feature is a concrete input-dependent number—not an abstract label.

The feature-transformation view

V VISUAL

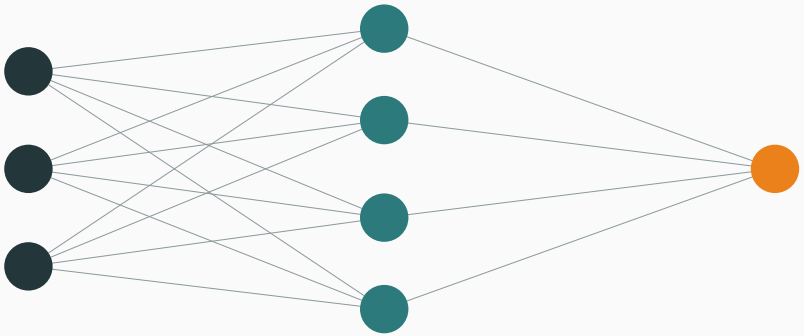


An MLP can learn a representation where classification becomes linear.

Binary uses one output node; K-class uses K

V VISUAL

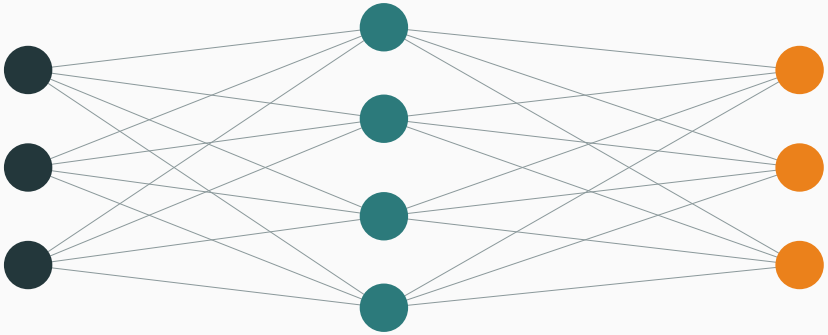
BINARY CLASSIFICATION



d inputs m hidden units 1 output node

$$z \in \mathbb{R} \to p = \sigma(z)$$

MULTI-CLASS CLASSIFICATION



d inputs m hidden units K output nodes

$$\boldsymbol{z} \in \mathbb{R}^K \to \boldsymbol{p} = \mathrm{softmax}(\boldsymbol{z})$$

Binary: 1 node gives p and $1-p$. Multi-class: K nodes give K softmax probabilities.

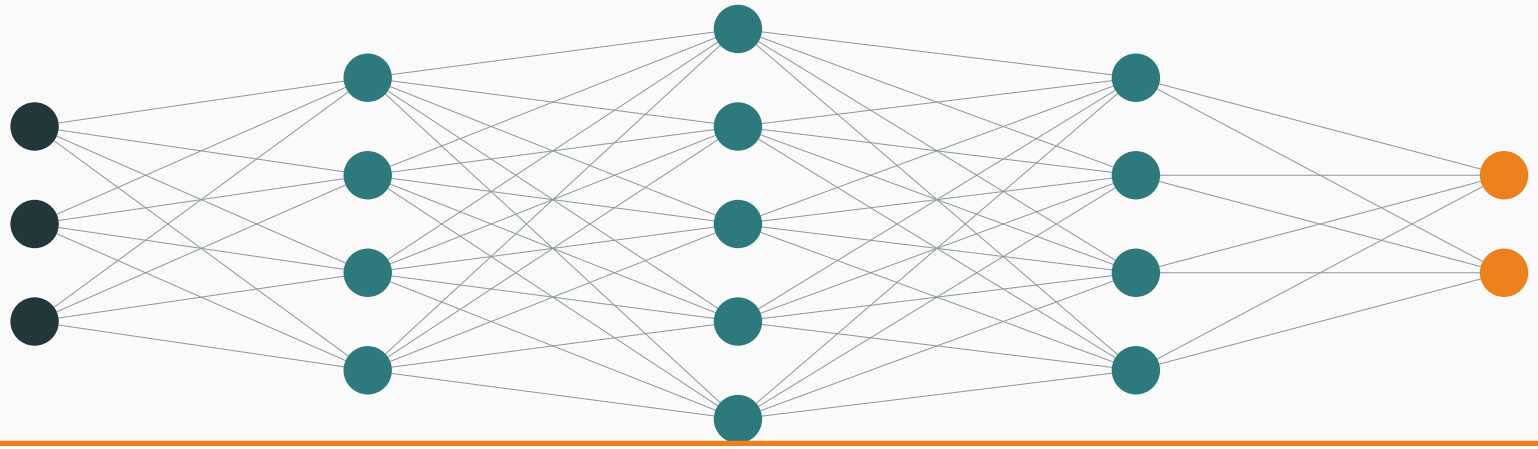
Depth stacks several hidden layers

V VISUAL

$$h^{(1)} = \varphi(W_1 x + b_1)$$

$$h^{(\ell)} = \varphi\big(W_\ell h^{(\ell-1)} + b_\ell\big), \quad \ell = 2, \ldots, L-1$$

$$z = W_L h^{(L-1)} + b_L$$



Each hidden layer transforms the previous representation. The output layer then produces task-specific scores or predictions.

Universal approximation

Universal approximation asks a precise question

Q QUESTION

Choose:

- a **continuous target** f;
- a **closed, bounded input region** D;
- an error tolerance $\varepsilon > 0$.

Can a finite neural network g make

$$|f(x) - g(x)| < \varepsilon$$

for **every** $x \in D$?

Universal-approximation theorem

With a suitable nonlinear activation, a one-hidden-layer network can approximate every continuous f on D as closely as requested.

This first says a network **exists**. It does not yet say that training will find it.

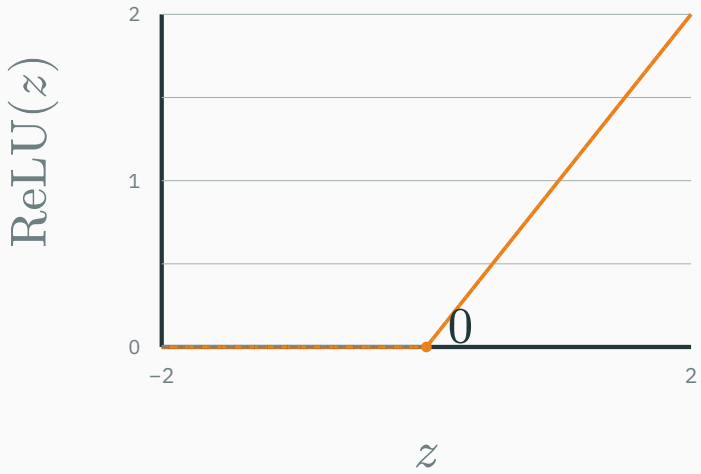
We will build the intuition with ReLU hinges, then read the formal claim carefully.

A ReLU contributes one hinge

V VISUAL

$$\varphi(z) = \max(0, z)$$

- $z < 0$: output is zero;
- $z > 0$: output grows linearly;
- at $z = 0$: the slope changes—a **hinge**.



A hidden layer combines many shifted hinges into a piecewise-linear function.

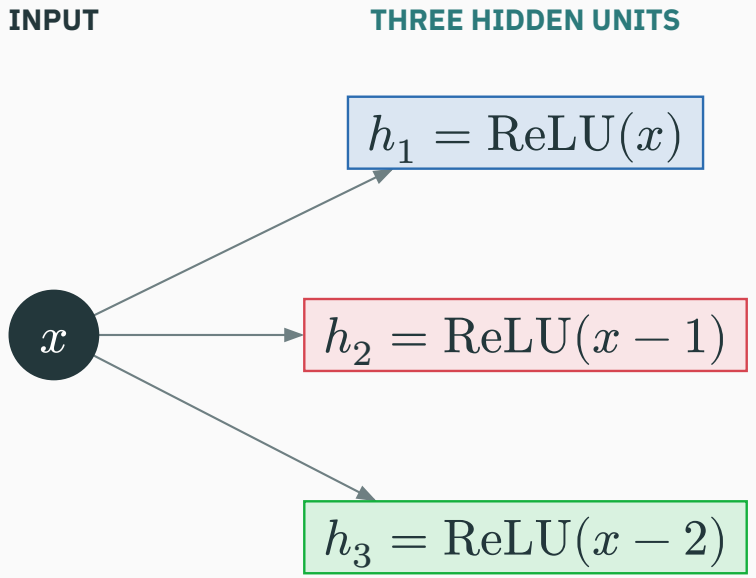
Step 1: one hidden layer creates three shifted ReLUs

V VISUAL

Unit 1 uses weight 1 and bias 0: $u_1 = 1x + 0, \quad h_1 = \mathrm{ReLU}(u_1) = \mathrm{ReLU}(x)$.

Unit 2 uses weight 1 and bias -1: $u_2 = 1x - 1, \quad h_2 = \mathrm{ReLU}(u_2) = \mathrm{ReLU}(x - 1)$.

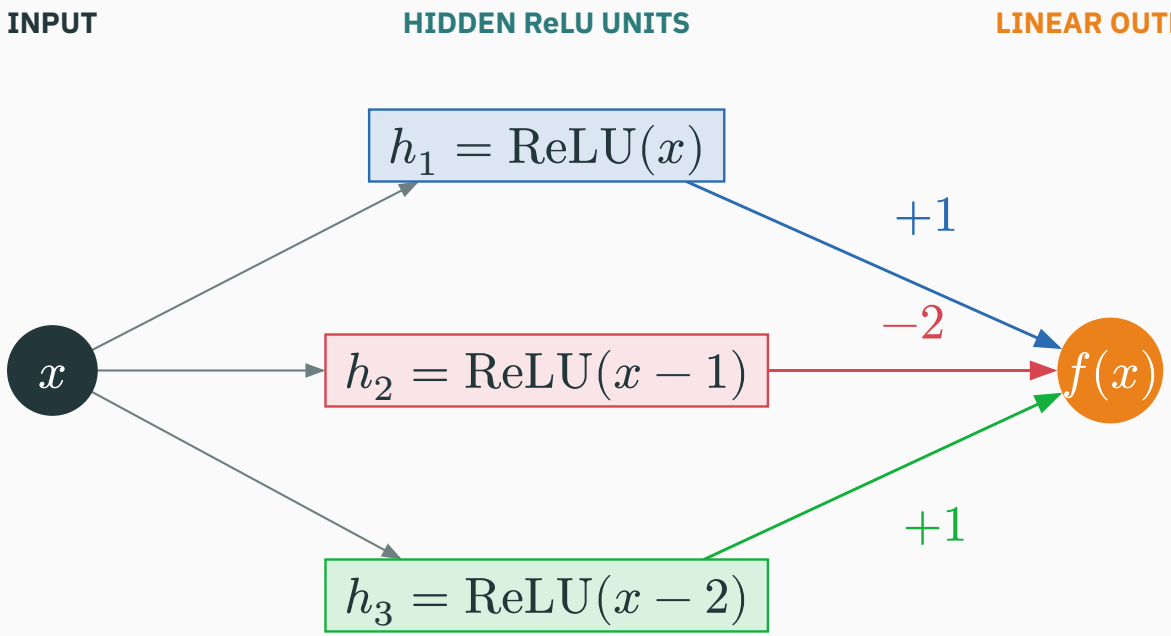
Unit 3 uses weight 1 and bias -2: $u_3 = 1x - 2, \quad h_3 = \mathrm{ReLU}(u_3) = \mathrm{ReLU}(x - 2)$.



The hidden biases move the three hinge locations to $x = 0, 1, 2$. No factor -2 has appeared yet.

Step 2: the output layer supplies the coefficients

V VISUAL



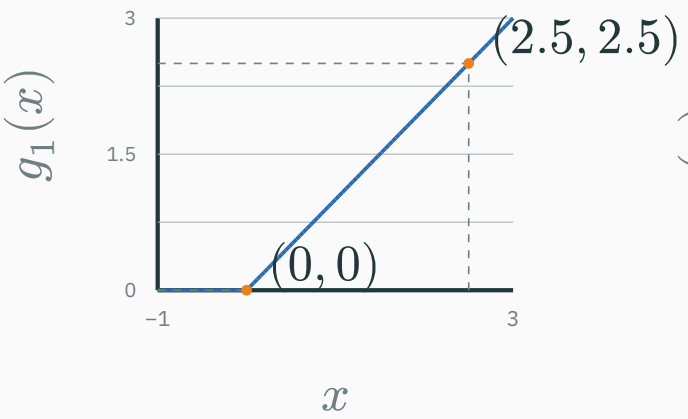
$$\boldsymbol{h} = \mathrm{ReLU}\left(\begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} x + \begin{pmatrix} 0 \\ -1 \\ -2 \end{pmatrix} \right), \quad f(x) = \begin{pmatrix} 1 & -2 & 1 \end{pmatrix} \boldsymbol{h}.$$

The -2 is an output-layer weight: the second hidden activation is multiplied by -2 only after ReLU.

Step 3: inspect the output contributions separately

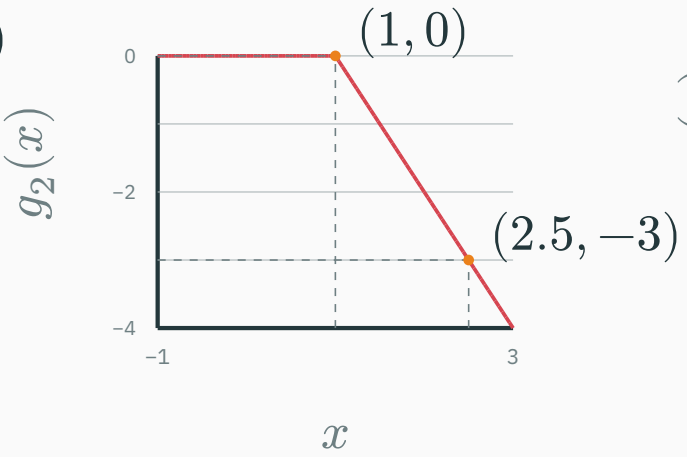
V VISUAL

$g_1(x) = (+1)h_1$



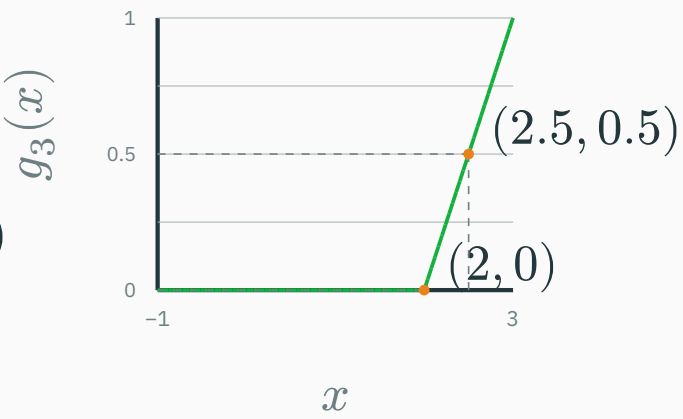
hinge at $x = 0$

$g_2(x) = (-2)h_2$



$h_2 \geq 0$; output weight -2 flips it

$g_3(x) = (+1)h_3$



hinge at $x = 2$

Hidden activations h_j are the features; $g_j = v_j h_j$ are their signed contributions to the output.

Step 4: add the active terms on each interval

D DERIVATION

$$f(x) = h_1 - 2h_2 + h_3 = \mathrm{ReLU}(x) - 2\mathrm{ReLU}(x-1) + \mathrm{ReLU}(x-2).$$

Input region	h_1	$-2h_2$	h_3	Sum $f(x)$
$x < 0$	0	0	0	0
$0 \le x < 1$	x	0	0	x
$1 \le x < 2$	x	$-2(x-1)$	0	$2 - x$
$x \ge 2$	x	$-2(x-1)$	$x - 2$	0

The output rises as x, falls as $2 - x$, then returns to zero: a tent.

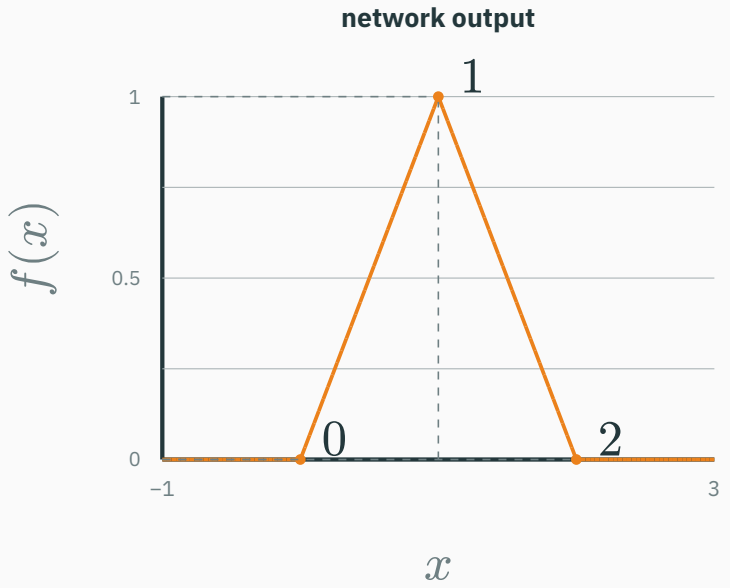
Step 5: the weighted sum is the tent

V VISUAL

Hidden layer $\boldsymbol{h}(x) = (\mathrm{ReLU}(x), \mathrm{ReLU}(x-1), \mathrm{ReLU}(x-2))$.

Linear output layer $f(x) = \begin{pmatrix} 1 & -2 & 1 \end{pmatrix} \boldsymbol{h}(x)$.

The network has **three hidden nodes** and **one output node**. The output node performs no ReLU here; it only takes a weighted sum.



Hidden units create reusable nonlinear features; the output layer chooses their signs and strengths.

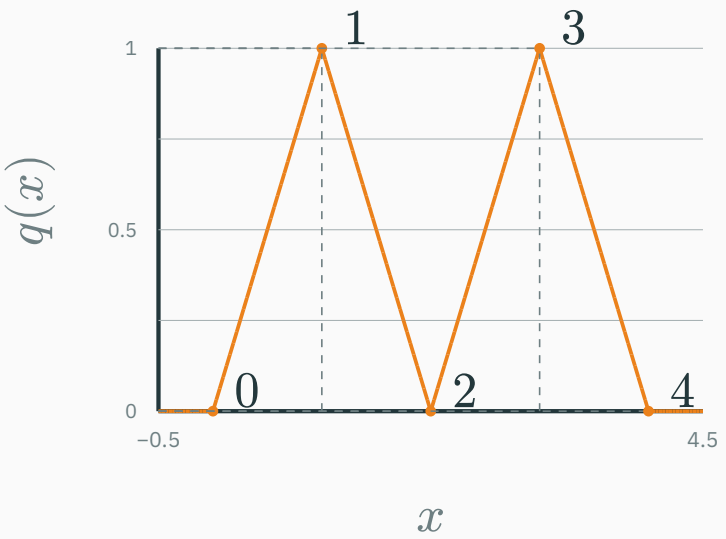
Five hinges create a two-peak function

V VISUAL

A slightly richer weighted sum:

$$q(x) = \mathrm{ReLU}(x) - 2\mathrm{ReLU}(x-1)$$
$$+2\mathrm{ReLU}(x-2) - 2\mathrm{ReLU}(x-3)$$
$$+ \mathrm{ReLU}(x-4).$$

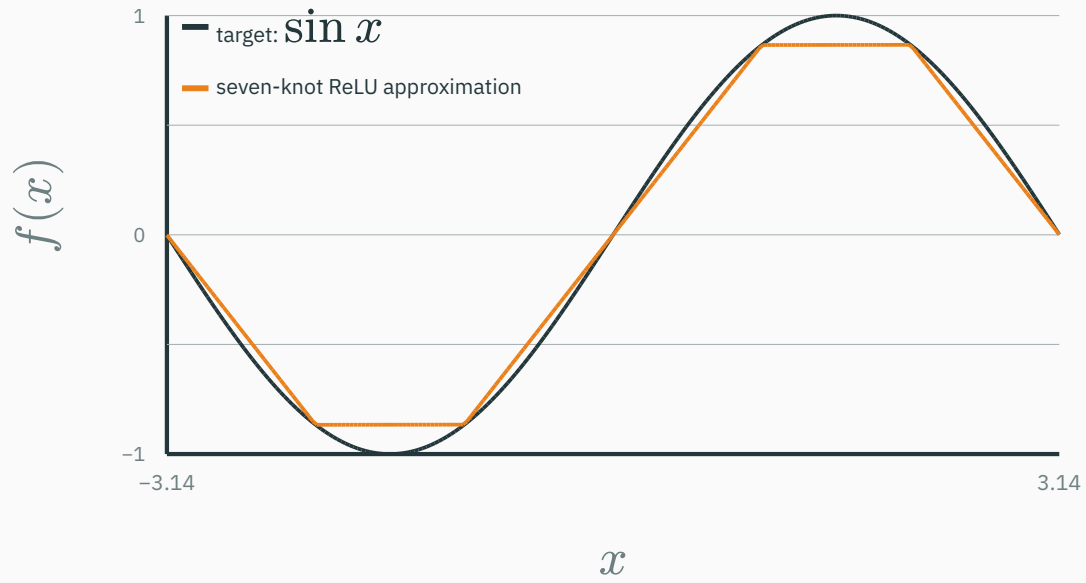
Each shifted unit changes the slope at one breakpoint.



More hidden units → more breakpoints → a more detailed piecewise-linear function.

More breakpoints can follow a smooth curve

V VISUAL



The orange approximation is still made only of straight segments; its corners are the hinges.

Adding hidden units adds breakpoints and reduces the approximation gap.

Learned ReLU units are the theorem's building blocks

D DERIVATION

For a one-dimensional input, a trained hidden unit has the form

$$h_{j(x)} = \mathrm{ReLU}(w_j x + b_j), \quad g(x) = c + \sum_j v_j h_{j(x)}.$$

j	hinge t_j	w_j	b_j	output v_j	signed component $v_j h_{j(x)}$
1	0	1	0	$+1$	$+\mathrm{ReLU}(x)$
2	1	1	-1	-2	$-2\mathrm{ReLU}(x-1)$
3	2	1	-2	$+1$	$+\mathrm{ReLU}(x-2)$

Training learns hinge locations and signed contributions; universal approximation says enough such units can make the remaining gap arbitrarily small.

Interactive: inspect the hidden units, not only the final curve

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/dl-teaching/interactives/relu-function-builder.html

Recreate one hinge, the three-unit tent, the five-unit two-peak function, and a piecewise-linear approximation to $\sin x$.

Translate every control into a parameter:

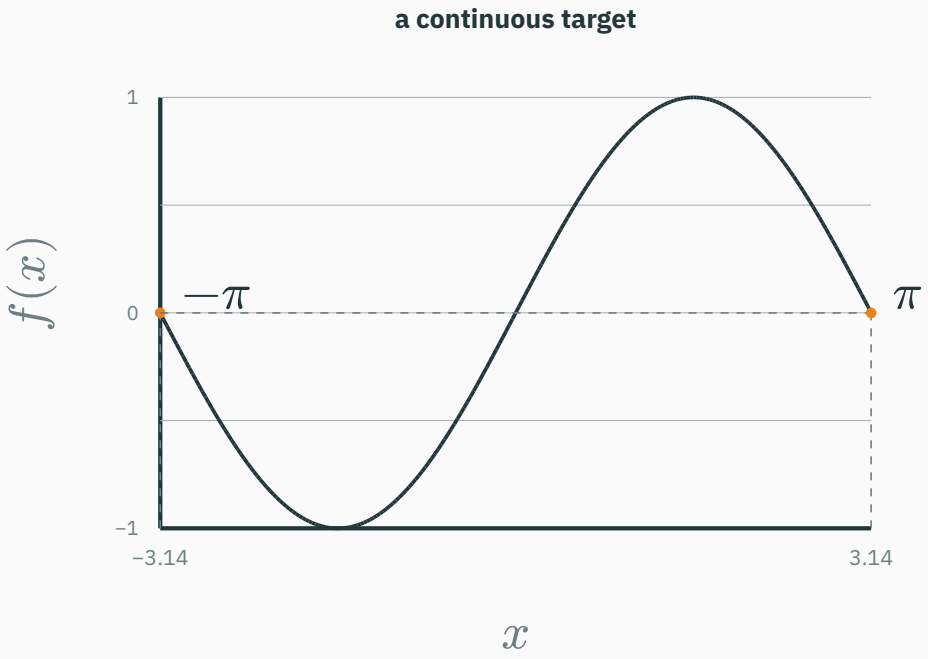
- hinge location $t_j \to b_j = -w_j t_j$;
- hidden slope $w_j \to$ orientation/ scale;
- output weight $v_j \to$ sign and slope change;
- show components $\to v_j h_{j(x)}$ and their sum.

For each preset, read off the **set of units** (w_j, b_j, v_j) first; then verify that their coloured signed

The builder is a parameter-level view of the same finite network used in the theorem.

First fix the target and the input region

V VISUAL



Continuous means that the target has no jumps.

In one dimension, a **compact domain** can be a closed, bounded interval such as

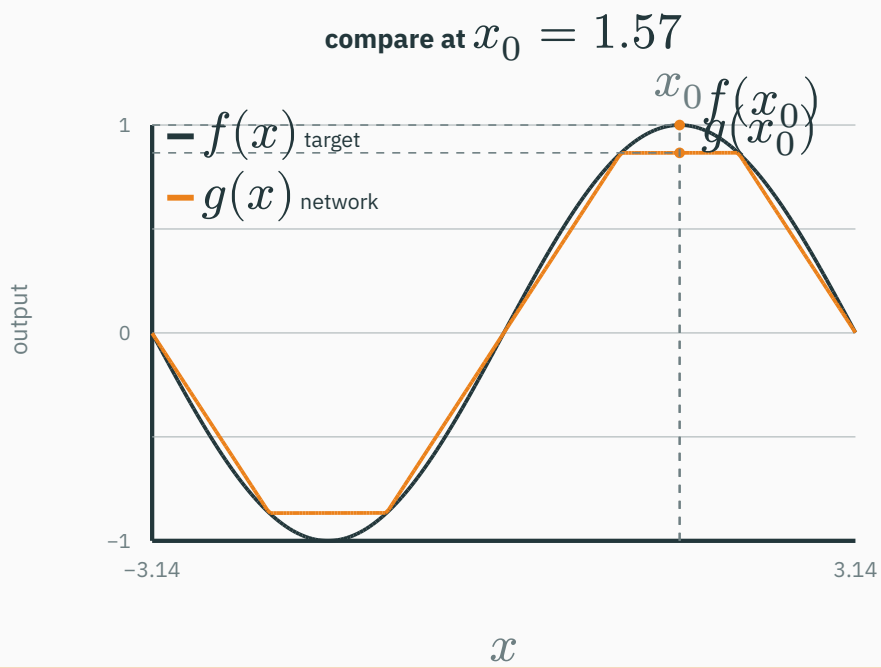
$$D = [-\pi, \pi].$$

We ask the network to match f **on** D; the theorem makes no claim outside that chosen region.

Approximation begins with one target function on one specified input region.

First measure the vertical gap at one input

V VISUAL



At one selected input x_0:

$$e(x_0) = |f(x_0) - g(x_0)|.$$

Here,

$$f(1.57) \approx 1.00,$$

$$g(1.57) \approx 0.87,$$

$$e(1.57) \approx 0.13.$$

Pointwise error is the vertical

Universal approximation requires controlling this gap at every input—not just at one convenient point.

“sup” means the worst gap over the whole region

V VISUAL

$\sup_{x \in D} |f(x) - g(x)|$

sup means **supremum**: the smallest number at least as large as every pointwise error.

Here the error is continuous and $D = [-\pi, \pi]$ is closed and bounded, so a highest error exists. In this plot, read **sup** as **maximum**.

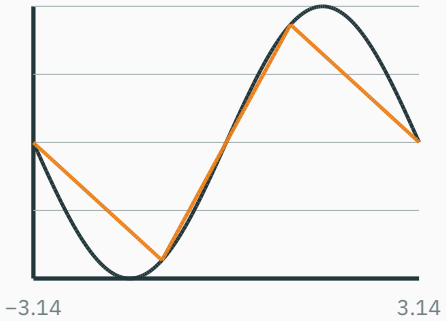
$\sup < \varepsilon$: no input in D crosses the tolerance.

The entire orange error curve stays below $\varepsilon = 0.05$, so the uniform guarantee is satisfied.

More hinges reduce the largest approximation error

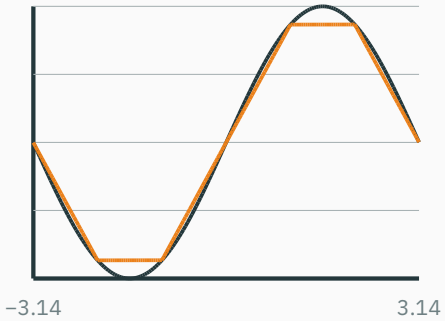
V VISUAL

4 knots



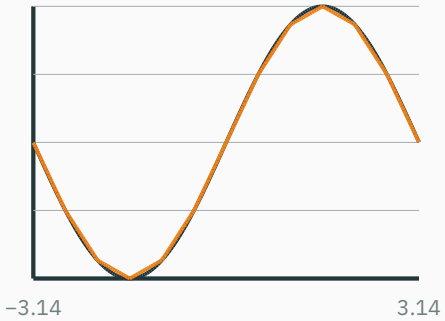
largest error ≈ 0.44

7 knots



largest error ≈ 0.13

13 knots

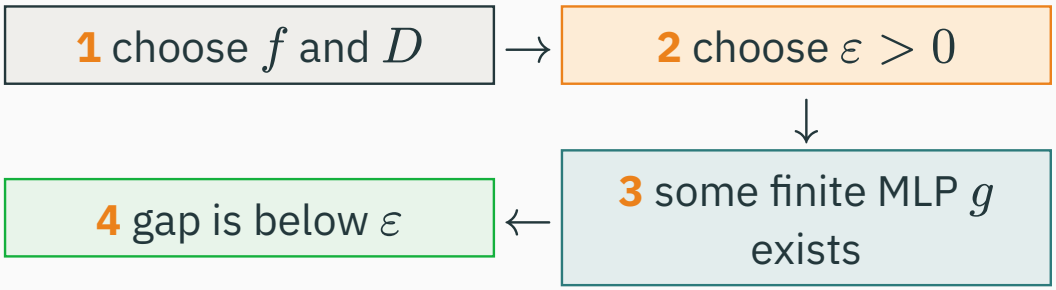


largest error ≈ 0.033

More hidden units provide more breakpoints, so a piecewise-linear network can follow finer detail.

Read the universal-approximation claim in order

D DERIVATION



The order matters.

For every ε: we may demand any positive tolerance.

There exists g: the network can change when we demand a smaller error.

$$\forall \varepsilon > 0, \exists \,\mathrm{MLP}\ g$$

$$\text{such that} \quad \sup_{x \in D} |f(x) - g(x)| < \varepsilon.$$

This is a statement about representational capacity—not yet about how to find the network.

Existence does not guarantee learnability

★ OPTIONAL

Universal approximation does **not** promise:

- that training will **find** that function;
- that **finite data** is enough;
- that a **small** network suffices;
- that it will **generalize**.

expressivity $\neq$ trainability $\neq$ generalization

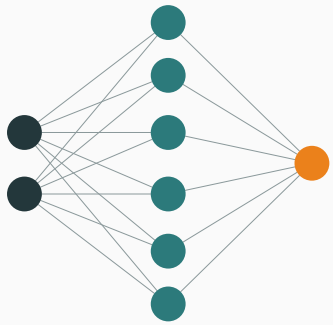
Depth versus width

Width and depth enlarge a network differently

V VISUAL

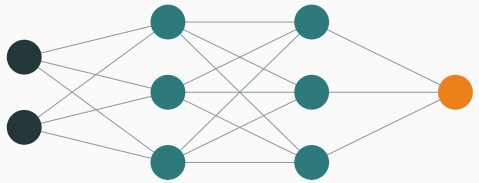
WIDER

one hidden layer · six parallel features



DEEPER

two hidden layers · three features per stage



Width adds parallel features; depth adds successive transformations. Universal approximation does not say width is always the most efficient organization.

Width buys more basis functions at one stage

V VISUAL

For a one-hidden-layer ReLU network,

$$g(x) = c + \sum_{j=1}^{m} v_j \, \mathrm{ReLU}(w_j x + b_j).$$

Increasing width m adds:

- another learned hinge;
- another signed component $v_j h_j$;
- potentially another breakpoint.

Width 3 → three hinges → one tent

$$\mathrm{ReLU}(x) - 2\mathrm{ReLU}(x-1) + \mathrm{ReLU}(x-2)$$

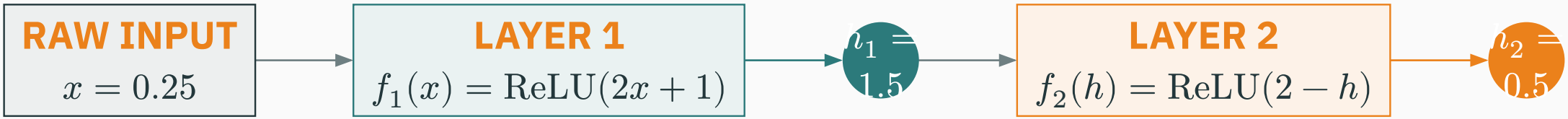
Greater width → more hinges → finer curve

$g(x) \approx \sin x$ on $[-\pi, \pi]$

Width gives a larger dictionary of features in one layer—the mechanism used in our universal-approximation construction.

Depth means feeding one learned result into the next

V VISUAL



$$h_2 = f_2(f_1(x)) = \mathrm{ReLU}(2 - \mathrm{ReLU}(2x + 1)).$$

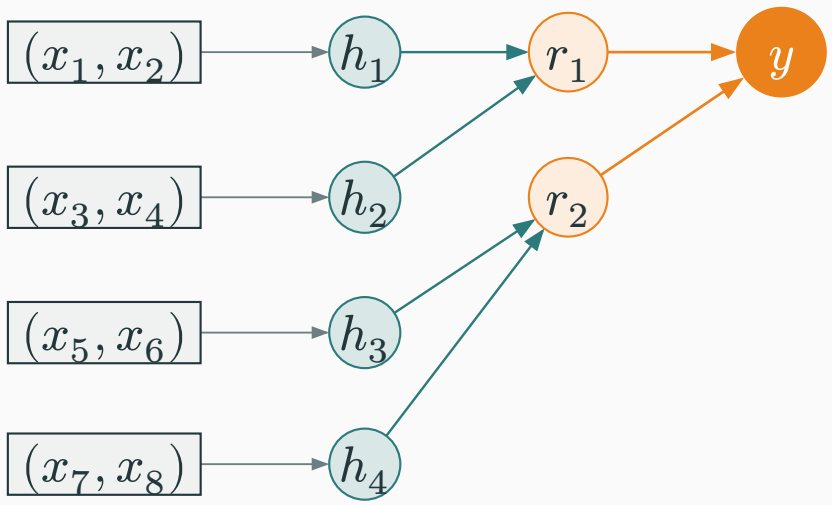
Layer 1 turns the raw input into the intermediate feature h_1.

Layer 2 never sees raw x directly here; it transforms the learned value h_1.

Composition is sequential reuse: $f = f_2 \circ f_1$. This is what added depth changes.

Depth can reuse intermediate computations

V VISUAL



Three stages

$$h_1 = q(x_1, x_2), \ldots, h_4 = q(x_7, x_8)$$

$$r_1 = q(h_1, h_2), \quad r_2 = q(h_3, h_4)$$

$$y = q(r_1, r_2)$$

Every small result is computed once, passed forward, and reused by the next stage.

When the target has a compositional structure, depth can mirror it; a flat layer must model the full interaction in one step.

Concrete feature hierarchies across three tasks

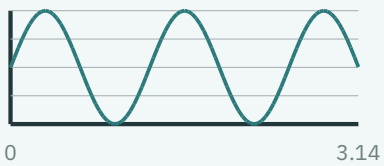
V VISUAL

8

VISION

pixels → curved strokes → two loops → digit 8

edge unit = 2.1 → upper-loop unit = 0.8



AUDIO

samples → tones/onsets → phoneme → word

440 Hz unit = 0.82 → onset unit is on

not good

TEXT

tokens → phrase pattern → sentiment → class

“not + good” unit = 1.7 → negative evidence

These labels are interpretations of learned numeric responses; later layers combine earlier responses into task-relevant evidence.

Depth and width are complementary design choices

D DERIVATION

Question	Increase width	Increase depth
What is added?	More parallel features in a layer.	More successive feature transformations.
Natural picture	A larger basis / more ReLU hinges.	A pipeline or hierarchy of reusable parts.
Useful when...	Many patterns must be detected at the same abstraction level.	The target is naturally compositional or hierarchical.
Main cost	Large activations and weight matrices within a layer.	Longer forward/backward paths and harder optimization.

There is no universally best shape. Architecture, data, optimization, and compute budget interact; compare models under a meaningful resource budget.

Modern networks usually use both: enough width per stage and enough depth to compose stages.

Interactive: compare width and depth on the same task

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/mlp-decision-boundary

Train a linear classifier or a small MLP on **XOR / circles / spirals / moons**. Compare one versus two hidden layers, then adjust width while watching the learned boundary. Controls: dataset · linear/MLP · hidden width · one/two layers · activation · train/reset/resample.

Hold the dataset fixed. First add width within one layer; then redistribute a similar number of hidden units across two layers. Which boundary appears sooner and which trains more reliably?

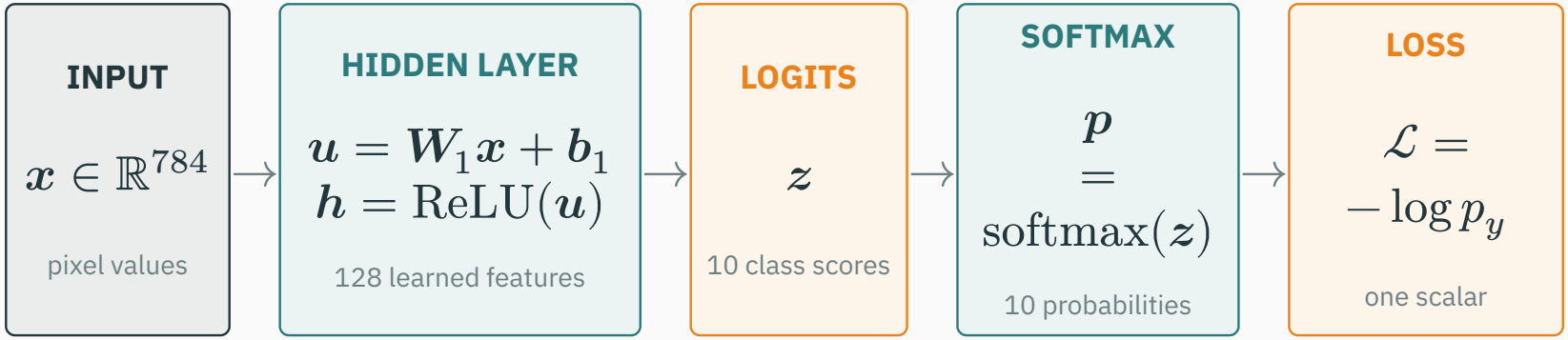
The comparison is empirical: width and depth change both representational structure and optimization behaviour.

Connecting to deep-learning practice

A forward pass transforms one input, step by step

V VISUAL

Example: classify one 28×28 image — 784 pixel values.



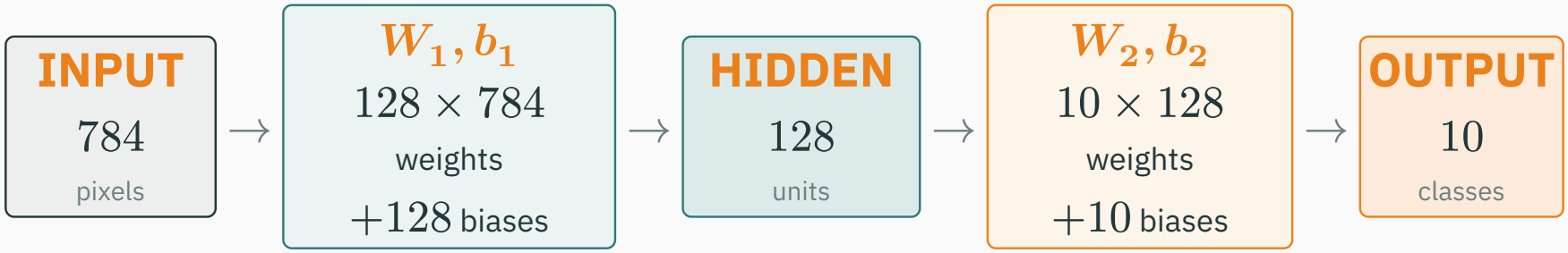
The true class y selects its predicted probability p_y; a smaller p_y gives a larger loss.

A forward pass computes a prediction and its loss; it does not update the weights.

Count parameters by following the same network

D DERIVATION

Every connection has a **weight**; every receiving unit has a **bias**.



Input → hidden

weights: $128 \times 784 = 100{,}352$

biases: 128

subtotal: $100{,}480$

Hidden → output

weights: $10 \times 128 = 1{,}280$

biases: 10

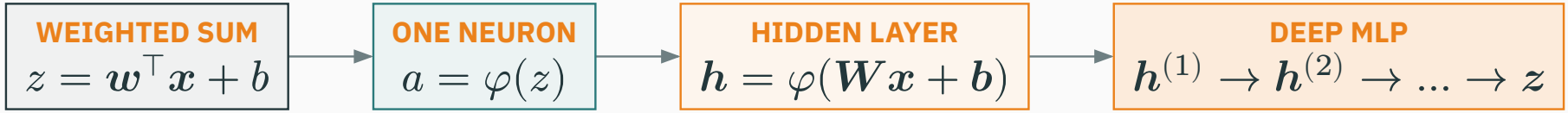
subtotal: $1{,}290$

Layer ℓ: $d_\ell d_{\ell-1}$ weights $+ d_\ell$ biases.

total $= 100{,}480 + 1{,}290 = 101{,}770$ **trainable parameters**

From one weighted sum to an MLP

V VISUAL



Stage	Representation	What it adds
Weighted sum	$z = w^\top x + b$	One line or plane; captures a global linear trend.
One neuron	$a = \varphi(z)$	One nonlinear feature: a hinge, gate, or score.
Hidden layer	$h = \varphi(Wx + b)$	Many learned features can bend or partition the input space.
Deep MLP	$h^{(\ell)} = \varphi(W_\ell h^{(\ell-1)} + b_\ell)$	Layers compose features into a hierarchical representation.

The key addition is not a new loss; it is a learned nonlinear representation.

The decisive change is the learned representation

Input x

↓

learned nonlinear features $h^{(1)}, \ldots, h^{(L-1)}$

↓

familiar Lecture 1 output model and loss

The output loss can stay the same while hidden layers make the representation nonlinear.

We now have $x \to f_{\boldsymbol{\theta}}(\boldsymbol{x}) \to \mathcal{L}$.

Lecture 3: how do we compute $\nabla_{\boldsymbol{\theta}} \mathcal{L}$ efficiently?

computation graphs · the chain rule · backpropagation · automatic differentiation